Introduction

What Is a Computer?

A computer is an electronic device that stores and processes data.

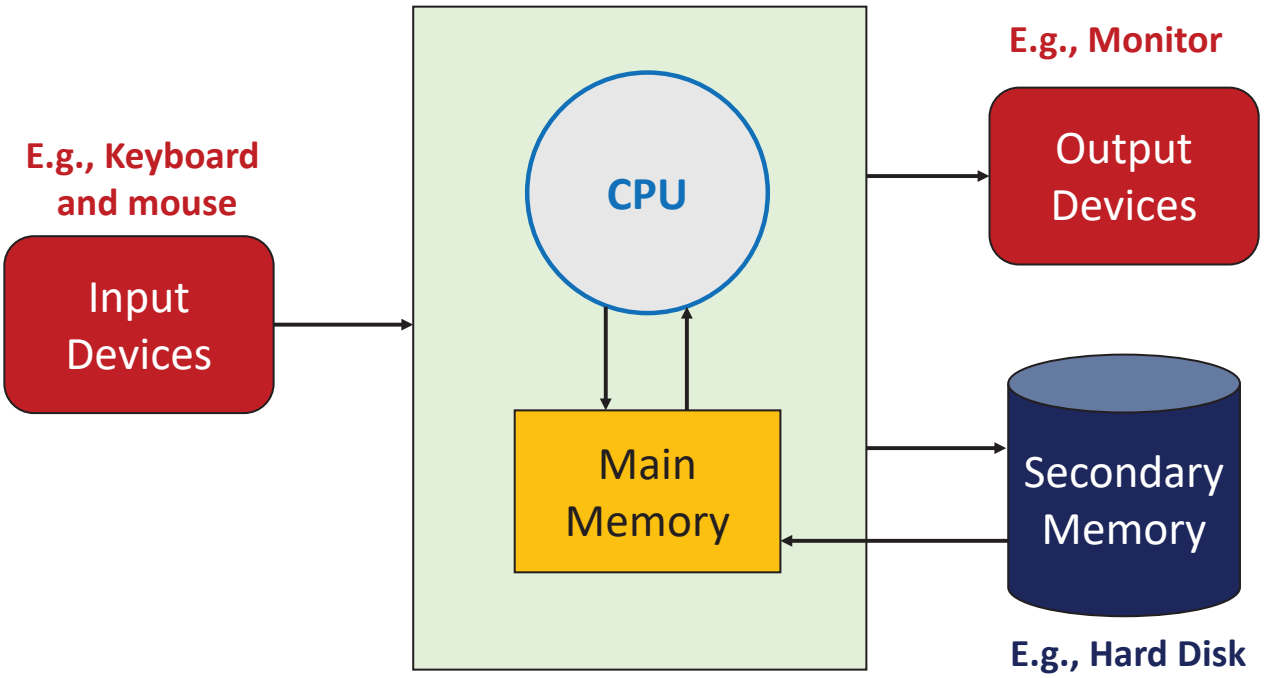
In general, hardware comprises the visible, physical elements of a computer, and software provides the invisible instructions that control the hardware and make it perform specific tasks.

Knowing computer hardware isn't essential to learning a programming language, but it can help you understand the effects that a program's instructions have on a computer and its components.

A computer consists of the following major hardware components:

- Central processing unit (CPU)
- Memory (main memory)
- Storage devices (such as disks and CDs)
- Input devices (such as the mouse and keyboard)
- Output devices (such as monitors and printers)
- Communication devices (such as modems and network interface cards)

Computer major hardware components:



Computer Languages and Applications (GEN241) Introduction

What is a computer program ?

A computer program is a set of detailed, step-by-step instructions written in a specific programming language for a computer to execute a particular task or solve a problem.

Why we need computer programming ?

We need computer programming to automate repetitive tasks, solve complex problems, drive innovation and economic growth, and create digital tools like software, apps, and websites.

How to write a computer program ?

Computers do not understand human languages; so programs must be written in languages computers can understand.

There are hundreds of programming languages, developed to make the programming process easier for people. However, all programs must be converted into instructions that the computer can execute.

To write a computer program you have to decide which programming language you are going to use and follow the syntax of that language.

Programming Languages

Programming languages are broadly categorized into three levels;

- **Lowest Level: Machine Language**
- **Low-Level: Assembly Language**
- **High Level: Programming languages**

Machine language, also known as machine code, is the lowest level programming language that a computer can directly understand. It handles binary (0 & 1). It directly interacts with system.

Machine language is extremely difficult for human beings to understand as it comprises combination of zeros and ones.

These are samples of instructions written in machine code;

```
0001111100001010101
0011100111001101010
0101010101010000000
1010101010101010101
```


Programming in machine language is a tedious process and very difficult to read and modify.

For this reason, Assembly Languages were developed in the early days of computing as an alternative to machine languages.

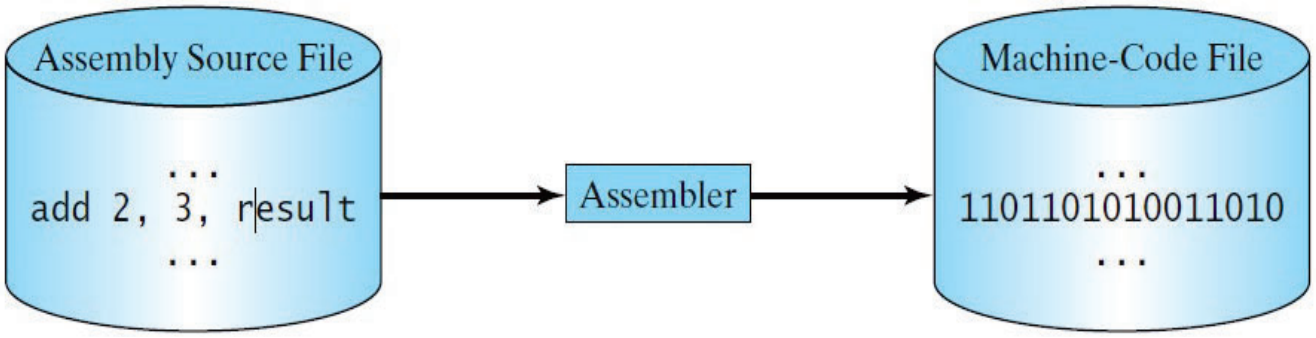
They were introduced to reduce the complexity of programming in machine code and make it more convenient for programmers.

Assembly languages use a short descriptive word, known as a *mnemonic*, to represent each of the machine-language instructions.

The following statements are samples of instruction code written in an Assembly language along with their corresponding machine code;

Assembly Language	Machine Code
SUB AX, BX	0010101110000011
MOV CX, AX	100010111001000
MOV DX, 0	10111010000000000000000

Because the computer cannot execute assembly code, another program - called an **Assembler** - is used to translate assembly-language programs into machine code.



Writing a program code in assembly language is easier than in machine language. However, it is still tedious.

Assembly language is referred to as a low-level language, because it is still close in nature to machine language and is **machine dependent**.

High-Level Languages

In the 1950s, a new generation of programming languages known as high-level languages emerged. They are **platform-independent**, which means that you can write a program in a high-level language and run it in different types of machines.

High-level languages are English-like and easy to learn and use. The instructions written in a high-level programming language are called program statements.

For example, in most high-level languages, a statement that computes the area of a circle with a radius of 5 can be written as follows:

```
area = 5 * 5 * 3.1415
```

There are many high-level programming languages, and each was designed for a specific purpose. The following Table lists some popular ones.

Language	Description
Ada	Named for Ada Lovelace, who worked on mechanical general-purpose computers. The Ada language was developed for the Department of Defense and is used mainly in defense projects.
BASIC	Beginner's All-purpose Symbolic Instruction Code. It was designed to be learned and used easily by beginners.
C	Developed at Bell Laboratories. C combines the power of an assembly language with the ease of use and portability of a high-level language.
C++	C++ is an object-oriented language, based on C.
C#	Pronounced "C Sharp." It is a hybrid of Java and C++ and was developed by Microsoft.
COBOL	COmmon Business Oriented Language. It is used for business applications.
FORTRAN	FORmula TRANslation. It is popular for scientific and mathematical applications.
Java	Developed by Sun Microsystems, now part of Oracle. It is widely used for developing platform-independent Internet applications.
Pascal	Named for Blaise Pascal, who pioneered calculating machines in the seventeenth century. It is a simple, structured, general-purpose language primarily used for teaching programming.
Python	A simple general-purpose scripting language good for writing short programs.
Visual Basic	Visual Basic was developed by Microsoft and it enables the programmers to rapidly develop graphical user interfaces.

Computer Languages and Applications (GEN241) Introduction

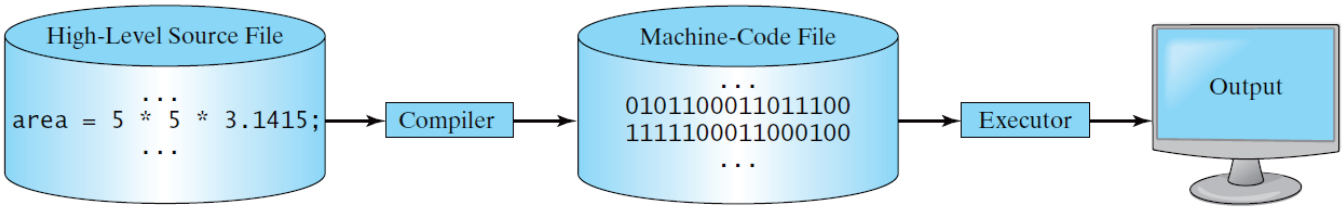
A program written in a high-level language is called a source program or source code.

Because a computer cannot execute a source code, a source program must be translated into machine code for execution.

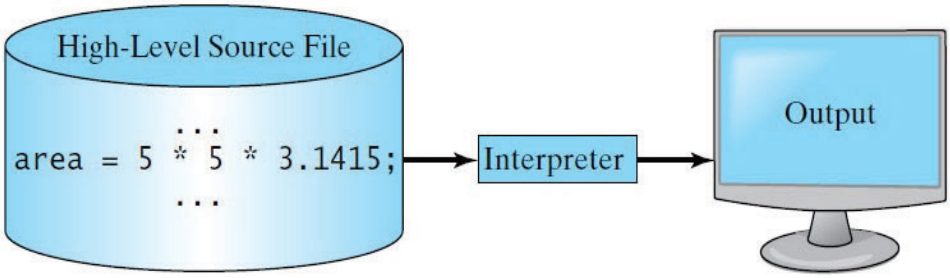
According to the in which the source code is translated into machine, programming languages are traditionally classified into two categories: Compiled or Interpreted.

In compiled languages, the entire source code is translated into machine code (binary) by a compiler before execution. This process produces a standalone executable file that can be executed directly by the computer's hardware. Compiled languages runs faster than interpreted ones.

It should be noted that compiled languages runs faster than interpreted ones.



Interpreted languages do not require an upfront translation step. Instead, an interpreter reads the source code line-by-line, translating and executing it in real-time while the program is running.



It should be noted that C++ is a compiled programming language.

C++ Programming Language

The present course is focused on learning computer programming with C++ programming language.

C++ is a general-purpose, object-oriented programming language.

C++ was invented by *Bjarne Stroustrup* at AT&T Bell Labs, beginning in 1979, with the initial goal of enhancing the C language by incorporating object-oriented programming features.

The first version, initially called "C with classes," was released in 1983 and later renamed C++. C++ has evolved significantly through standardization efforts and the addition of new features, becoming a widely used, general-purpose language.

Writing a Simple Program

First, how do we write a program for solving a problem?

In general, writing a program involves two steps;

- ❑ **Designing algorithm.**
- ❑ **Translating the algorithm into programming instructions, or code.**

Let's consider the simple problem of computing the area of a circle.

For this problem, the algorithm will be as follows;

- ❑ **Read in the circle's radius.**
- ❑ **Compute the area using the following formula:**

$$area = radius \times radius \times \pi$$

- ❑ **Display the result.**

Let us translate this algorithm into programming instructions;

Ex_0201.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6    double radius;
7    double area;
8
9    // Step 1: Read in radius
10   radius = 20;
11
12   // Step 2: Compute area
13   area = radius * radius * 3.14159;
14
15   // Step 3: Display the area
16   cout << "The area is " << area << endl;
17
18   return 0;
19 }
```

Details of this code will be explained in the lecture.

Reading Input from the Keyboard

In previous example, the radius is fixed in the source code. To use a different radius, you have to modify the source code and recompile it. Obviously, this is not convenient. Let us modify the last code as follows;

Ex_0202.cpp

```cpp
#include <iostream>
using namespace std;

int main()
{
  // Step 1: Read in radius
  double radius;
  cout << "Enter a radius: ";
  cin >> radius;

  // Step 2: Compute area
  double area = radius * radius * 3.14159;

  // Step 3: Display the area
  cout << "The area is " << area << endl;

  return 0;
}
```


Let solve the problem of finding the average value of three numbers. The program code will be as follows;

Ex_0203.cpp

```cpp
#include <iostream>
using namespace std;

int main()
{
  // Prompt the user to enter three numbers
  double number1, number2, number3;
  cout << "Enter three numbers: ";
  cin >> number1 >> number2 >> number3;

  // Compute average
  double average = (number1 + number2 + number3) / 3;

  // Display result
  cout << "The average of " << number1 << " " << number2
    << " " << number3 << " is " << average << endl;

  return 0;
}
```

Identifiers

Identifier is a user-defined name used to identify various program elements such as *variables*, *named constants* and *functions*.

Let us have a look on the code of the last example

Ex_0204.cpp

```
#include <iostream>
using namespace std;

int main()
{
  // Prompt the user to enter three numbers
  double number1, number2, number3;
  cout << "Enter three numbers: ";
  cin >> number1 >> number2 >> number3;

  // Compute average
  double average = (number1 + number2 + number3) / 3;

  // Display result
  cout << "The average of " << number1 << " " << number2
    << " " << number3 << " is " << average << endl;

  return 0;
}
```

As you see in the last example, number1, number2, number3, average and so on are the names of things that appear in the program.

In programming terminology, such names are called *identifiers*. All identifiers must obey the following rules:

- An identifier is a sequence of characters comprising letters, digits, and underscores.
- An identifier must start with a letter or an underscore; it cannot start with a digit.
- An identifier cannot be a reserved word. A list of C++ keywords is given below.
- An identifier can be of any length, but your C++ compiler may impose restriction. Use identifiers of 31 characters or fewer to ensure portability.

For example, area and radius are legal identifiers, whereas 2A and d+4 are illegal identifiers because they do not follow the rules. The compiler detects illegal identifiers and reports syntax errors.

C++ Keywords

The following keywords are reserved for use by the C++ language. They should not be used for anything other than their predefined purposes in C++.

asm	do	inline	short	typeid
auto	double	int	signed	typename
bool	dynamic_cast	long	sizeof	union
break	else	mutable	static	unsigned
case	enum	namespace	static_cast	using
catch	explicit	new	struct	virtual
char	extern	operator	switch	void
class	false	private	template	volatile
const	float	protected	this	wchar_t
const_cast	for	public	throw	while
continue	friend	register	true	
default	goto	reinterpret_cast	try	
delete	if	return	typedef	

Variables

Variables are used to represent values that may be changed in the program.

Let us have a look on a last example for computing the area of a circle;

Ex_0205.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6    // Step 1: Read in radius
7    double radius;
8    cout << "Enter a radius: ";
9    cin >> radius;
10
11   // Step 2: Compute area
12   double area = radius * radius * 3.14159;
13
14   // Step 3: Display the area
15   cout << "The area is " << area << endl;
16
17   return 0;
18 }
```

In this program, radius and area are variables of the double-precision, floating-point type. You can assign any numerical value to radius and area.

Within the program code, variables may be reassigned new values. For example, in the following code, radius is initially 1.0 (line 2) and then changed to 2.0 (line 7), and area is set to 3.14159 (line 3) and then reset to 12.56636 (line 8).

```
1  // Compute the first area
2  radius = 1.0;                                    radius:  1.0
3  area = radius * radius * 3.14159;                  area:  3.14159
4  cout << "The area is " << area << " for radius " << radius;
5
6  // Compute the second area
7  radius = 2.0;                                    radius:  2.0
8  area = radius * radius * 3.14159;                  area:  12.56636
9  cout << "The area is " << area << " for radius " << radius;
```

The syntax for declaring a variable is

```
datatype variableName;
```

Here are some examples of variable declarations:

```
int count;           // Declare count to be an integer variable
double radius;       // Declare radius to be a double variable
double interestRate; // Declare interestRate to be a double variable
```

These examples use the data types int and double. Later you will be introduced to additional data types, such as short, long, float, char, and bool.

If variables are of the same type, they can be declared together, as follows:

```
datatype variable1, variable2,..., variablen;
```

The variables are separated by commas. For example,

```
int i, j, k; // Declare i, j, and k as int variables
```

Variables often have initial values. You can declare a variable and initialize it in one-step. Consider, for instance, the following code:

```
int count = 1;
```

This is equivalent to the next two statements:

```
int count;
count = 1;
```

You can also use shorthand to declare and initialize variables of the same type together. For example,

```
int i = 1, j = 2;
```

Assignment Statements and Assignment Expressions

An assignment statement designates a value for a variable.

In C++, the equal sign (=) is used as the *assignment operator*.

The syntax for assignment statements is as follows:

```
variable = expression;
```

An *expression* represents a computation involving values, variables, and operators that, taking them together, evaluates to a value. For example, consider the following code:

```
int y = 1;                  // Assign 1 to variable y
double radius = 1.0;        // Assign 1.0 to variable radius
int x = 5 * (3 / 2);        // Assign the value of the expression to x
x = y + 1;                  // Assign the addition of y and 1 to x
area = radius * radius * 3.14159; // Compute area
```

You can use a variable in an expression. A variable can also be used in both sides of the = operator. For example,

```
x = x + 1;
```

To assign a value to a variable, you must place the variable name to the left of the assignment operator. Thus, the following statement is wrong:

```
1 = x; // Wrong
```

Named Constants

A named constant is an identifier that represents a permanent value.

Here is the syntax for declaring a constant:

```
const datatype CONSTANTNAME = value;
```

A constant must be declared and initialized in the same statement.

The word const is a C++ keyword for declaring a constant.

For example,

```
const double PI = 3.14159;
```

Numeric Data Types and Operations

2.8.1 Numeric Types

Every data type has a range of values. The compiler allocates memory space for each variable or constant according to its data type.

Table 2.1 lists the numeric data types with their storage sizes and typical ranges.

	Type	Typical Bit Width	Typical Range
Integer Numeric Data Type	int	4bytes	-2147483648 to 2147483647
	unsigned int	4bytes	0 to 4294967295
	signed int	4bytes	-2147483648 to 2147483647
	short int	2bytes	-32768 to 32767
	unsigned short int	2bytes	0 to 65,535
	signed short int	2bytes	-32768 to 32767
	long int	8bytes	-9223372036854775808 to 9223372036854775807
	signed long int	8bytes	same as long int
	unsigned long int	8bytes	0 to 18446744073709551615
	long long int	8bytes	-(2^63) to (2^63)-1
	unsigned long long int	8bytes	0 to 18,446,744,073,709,551,615
Floating-Point Numeric Data Type	float	4bytes	Negative range: -3.4028235E+38 to -1.4E-45 Positive range: 1.4E-45 to 3.4028235E+38
	double	8bytes	Negative range: -1.7976931348623157E+308 to -4.9E-324 Positive range: 4.9E-324 to 1.7976931348623157E+308
	long double	12bytes	Negative range: -1.18E+4932 to -3.37E-4932 Positive range: 3.37E-4932 to 1.18E+4932

C++ uses three types for integers: short, int, and long. Each integer type comes in two flavors: signed and unsigned.

C++ uses three types for floating-point numbers: float, double, and long double. The double type is usually twice as big as float. So, the double is known as double precision, while float is single precision. The long double is even bigger than double. For most applications, using the double type is desirable.

For convenience, C++ defines constants INT_MIN, INT_MAX, LONG_MIN, LONG_MAX, FLT_MIN, FLT_MAX, DBL_MIN, and DBL_MAX in the <limits> header file. These constants are useful in programming. Run the following code and see what constant values are defined by your compiler:

Ex_0206.cpp

```cpp
#include <iostream>
#include <limits>
using namespace std;

int main()
{
  cout << "INT_MIN is " << INT_MIN << endl;
  cout << "INT_MAX is " << INT_MAX << endl;
  cout << "LONG_MIN is " << LONG_MIN << endl;
  cout << "LONG_MAX is " << LONG_MAX << endl;
  cout << "FLT_MIN is " << FLT_MIN << endl;
  cout << "FLT_MIN is " << FLT_MAX << endl;
  cout << "DBL_MIN is " << DBL_MIN << endl;
  cout << "DBL_MIN is " << DBL_MAX << endl;

  return 0;
}
```

When you run the above code it gives the following output

```
INT_MIN is -2147483648
INT_MAX is 2147483647
LONG_MIN is -2147483648
LONG_MAX is 2147483647
FLT_MIN is 1.17549e-38
FLT_MIN is 3.40282e+38
DBL_MIN is 2.22507e-308
DBL_MIN is 1.79769e+308

--------------------------------
Process exited after 0.1496 seconds with return value 0
Press any key to continue . . .
```

The following example displays the size of int, long, and double, and variables age and area on your machine.

Ex_0207.cpp

```cpp
#include <iostream>
using namespace std;

int main()
{
  cout << "The size of int: " << sizeof(int) << " bytes" << endl;
  cout << "The size of long: " << sizeof(long) << " bytes" << endl;
  cout << "The size of double: " << sizeof(double)
    << " bytes" << endl;

  double area = 5.4;
  cout << "The size of variable area: " << sizeof(area)
    << " bytes" << endl;

  int age = 31;
  cout << "The size of variable age: " << sizeof(age)
    << " bytes" << endl;

  return 0;
}
```

When you run the above code it gives the following output

```
The size of int: 4 bytes
The size of long: 4 bytes
The size of double: 8 bytes
The size of variable area: 8 bytes
The size of variable age: 4 bytes

--------------------------------
Process exited after 0.1867 seconds with return value 0
Press any key to continue . . .
```

Numeric Operators

Operator	*Name*	*Example*	*Result*
+	Addition	34 + 1	35
-	Subtraction	34.0 − 0.1	33.9
*	Multiplication	300 * 30	9000
/	Division	1.0 / 2.0	0.5
%	Modulus	20 % 3	2

Notes:

- When both operands of a division are integers, the result of the division is the quotient and the fractional part is truncated. For example, 5/2 yields 2, not 2.5, and –5/2 yields -2, not –2.5. To perform regular mathematical division, one of the operands must be a floating point number. For example, 5.0/2 yields 2.5.
- The % operator, known as *modulo* or *remainder* operator, works only with integer operands and yields the remainder after division. The left-hand operand is the dividend and the right hand operand the divisor. Therefore, 7%3 yields 1, 3%7 yields 3, 12%4 yields 0, 26%8 yields 2, and 20%13 yields 7.

Example: The following program obtains minutes and remaining seconds from an amount of time in seconds. For example, 500 seconds contains 8 minutes and 20 seconds.

Ex_0208.cpp

```
#include <iostream>
using namespace std;

int main()
{
  // Prompt the user for input
  int seconds;
  cout << "Enter an integer for seconds: ";
  cin >> seconds;
  int minutes = seconds / 60;
  int remainingSeconds = seconds % 60;
  cout << seconds << " seconds is " << minutes <<
    " minutes and " << remainingSeconds << " seconds " << endl;

  return 0;
}
```

Exponent Operations

The pow(a, b) function can be used to compute a^b. pow is a function defined in the cmath library. The function is invoked using the syntax pow(a, b) (i.e., pow(2.0, 3)) that returns the result of a^b (2^3). Here, a and b are parameters for the pow function and numbers 2.0 and 3 are actual values used to invoke the function. For example,

```
cout << pow(2.0, 3) << endl; // Display 8.0
cout << pow(4.0, 0.5) << endl; // Display 2.0
cout << pow(2.5, 2) << endl; // Display 6.25
cout << pow(2.5, -2) << endl; // Display 0.16
```

Note that some C++ compilers require that either a or b in pow(a, b) be a decimal value. Here we use 2.0 rather than 2.

Example The last program for computing the area of a circle is modified as follows;

Ex_0209.cpp

```
1  // Compute the Circle Area
2  #include <iostream>
3  #include<cmath>
4  using namespace std;
5
6  int main()
7  {
8      double radius;
9      double area;
10     const double PI=3.14159;
11
12     // Step 1: Read in the radius
13     radius = 20;
14
15     // Step 2: Compute the area
16     // area = radius*radius*3.14159;
17     area = pow(radius,2)*PI;
18
19     // Step 3: Display the area
20     cout << "The area is " << area << endl;
21 return 0;
22 }
```

Operator Precedence

Writing numeric expressions in C++ involves a straightforward translation of an arithmetic expression using C++ operators. For example, the arithmetic expression

$$\frac{3+4x}{5}-\frac{10(y-5)(a+b+c)}{x}+9\left(\frac{4}{x}+\frac{9+x}{y}\right)$$

can be translated into a C++ expression as

```
(3 + 4 * x) / 5 - 10 * (y - 5) * (a + b + c) / x +
9 * (4 / x + (9 + x) / y)
```

When more than one operator is used in an expression, the following operator precedence rule is used to determine the order of evaluation.

- Multiplication, division, and remainder operators are applied next. If an expression contains several multiplication, division, and remainder operators, they are applied from left to right.
- Addition and subtraction operators are applied last. If an expression contains several addition and subtraction operators, they are applied from left to right.

The following program converts a Fahrenheit degree to Celsius using the formula $\text{celsius} = \left(\frac{5}{9}\right)(\text{fahrenheit} - 32)$.

Ex_0210.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6    // Enter a degree in Fahrenheit
7    double fahrenheit;
8    cout << "Enter a degree in Fahrenheit: ";
9    cin >> fahrenheit;
10
11   // Obtain a celsius degree
12   double celsius = (5.0 / 9) * (fahrenheit - 32);
13
14   // Display result
15   cout << "Fahrenheit " << fahrenheit << " is " <<
16       celsius << " in Celsius" << endl;
17
18   return 0;
19 }
```

Augmented Assignment Operators

The operators +, -, *, /, and % can be combined with the assignment operator to form augmented operators. The following table shows how augmented operators work.

Operator	*Name*	*Example*	*Equivalent*
+=	Addition assignment	`i += 8`	`i = i + 8`
-=	Subtraction assignment	`i -= 8`	`i = i - 8`
*=	Multiplication assignment	`i *= 8`	`i = i * 8`
/=	Division assignment	`i /= 8`	`i = i / 8`
%=	Modulus assignment	`i %= 8`	`i = i % 8`

Increment and Decrement Operators

The increment (++) and decrement (--) operators are for incrementing and decrementing a variable by 1.

The ++ and -- are two shorthand operators for incrementing and decrementing a variable by 1.

For example, the following code increments i by 1 and decrements j by 1.

```
int i = 3, j = 3;
i++; // i becomes 4
j--; // j becomes 2
```

This is called *postincrement* or *postdecremen*

A *preincrement* and *predecrement* are also possible

```
int i = 3, j = 3;
++i; // i becomes 4
--j; // j becomes 2
```

Numeric Type Conversions

Floating-point numbers can be converted into integers using explicit casting.

Can you assign an integer value to a floating-point variable? Yes.

Can you assign a floating point value to an integer variable? Yes.

When assigning a floating-point value to an integer variable, the fractional part of the floating-point value is truncated (not rounded). For example:

```
int i = 34.7;       // i becomes 34
double f = i;       // f is now 34
double g = 34.3;    // g becomes 34.3
int j = g;          // j is now 34
```

Can you perform binary operations with two operands of different types? Yes.

If an integer and a floating-point number are involved in a binary operation, C++ automatically converts the integer to a floating-point value. So, 3*4.5 is the same as 3.0*4.5.

C++ also allows you to convert a value from one type to another explicitly by using a *casting operator*. The syntax is

```
static_cast<type>(value)
```

For example, the following statement

```
cout << static_cast<int>(1.7);
```
displays 1

```
cout << static_cast<double>(1) / 2;
```
displays 0.5

Example

Ex_0211.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6    // Enter purchase amount
7    double purchaseAmount;
8    cout << "Enter purchase amount: ";
9    cin >> purchaseAmount;
10
11   double tax = purchaseAmount * 0.06;
12   cout << "Sales tax is " << static_cast<int>(tax * 100) / 100.0;
13
14   return 0;
15 }
```

When you run the above code it gives the following output

```
Enter purchase amount: 197.55
Tax is 11.853
Sales tax is 11.85
--------------------------------
Process exited after 18.22 seconds with return value 0
Press any key to continue . . .
```